Clean Code:

Code is clean if it can be understood easily – by everyone on the team. Clean code can be read and enhanced by a developer other than its original author. With understandability comes readability, changeability, extensibility and maintainability.

General rules

1. Follow standard conventions.

```
class UserProfile:
    def get_user_name(self):
        return "Alice"
```

If you break this and name a class like `user_profile`, it may confuse other developers.

2. Keep it simple stupid. Simpler is always better. Reduce complexity as much as possible.

Bad	Good
if user.is_logged_in() == True: print("Welcome")	if user.is_logged_in(): print("Welcome")

3. Boy scout rule. Improve the code a little **whenever you touch it** — like cleaning up a campsite. Even small refactors make a big difference over time.
4. Always find the root cause. Always look for the root cause of a problem.

Symptom: Server crashes when uploading large files.
Temporary fix: Block large files.
Root cause: Memory leak in file parser.
Proper fix: Fix the memory leak.

Design rules

1. Keep configurable data at high levels.- Configuration (like file paths, timeouts, limits) should reside in top-level components (e.g., config files, environment variables), not hardcoded deep in the codebase.

Bad	Good

db.connect("localhost", 5432)	# config.py
	DB_HOST = "localhost"
	DB_PORT = 5432
	
	# db_connector.py
	import config
	connect_to_db(config.DB_HOST, config.DB_PORT)

2. Prefer polymorphism to if/else or switch/case.

Bad	Good

if shape.type == "circle":
  shape.draw_circle()
elif shape.type == "square":
  shape.draw_square()
	class Shape:
	  def draw(self): pass
	
	class Circle(Shape):
	  def draw(self): print("Drawing circle")
	
	class Square(Shape):
	  def draw(self): print("Drawing square")

3. Separate multi-threading code.- Isolate multi-threading or concurrency logic from business logic to reduce complexity and improve testability.
4. Prevent over-configurability. Avoid exposing too many options that make the system hard to use, test, or maintain. Only make what ***needs*** to be configurable.

Bad	Good
config = {"max_connections": 5, "min_connections": 1, "connection_growth_rate": 1.2, ...}	config = {"max_connections": 10}

5. Use dependency injection. Inject dependencies rather than creating them internally. This makes code more modular and testable.

Bad	Good

class Service:
  def __init__(self):
	class Service:
	  def __init__(self, db):

self.db = Database()	self.db = db

6. Follow the Law of Demeter. A class should know only its direct dependencies.

Bad	Good
user.get_profile().get_settings().get_theme()	user.get_theme() # Hide the chain of calls inside the user class

Understandability tips

1. Be consistent. Follow consistent naming, structure, and logic across your codebase. It reduces confusion and increases readability.
2. Use explanatory variable names.
3. Encapsulate boundary conditions. Handle edge/boundary conditions in one place so they don't clutter your main logic and are easier to manage or update.

```
# Encapsulated
def is_valid_age(age):
    return 0 <= age <= 120
```

4. Prefer dedicated value objects to primitive type. **Wrap primitive types** (like strings, ints) in objects when they represent specific concepts to give them meaning and behavior.
5. Avoid logical dependency. Don't write methods which works correctly depending on something else in the same class.
6. Avoid negative conditionals.

Bad	Good

if not is_user_authorized(user):
    return "Access denied"
	if is_user_unauthorized(user):
	    return "Access denied"

Names rules

1. Choose descriptive and unambiguous names.
2. Make meaningful distinctions.
3. Use pronounceable names.
4. Use searchable names.
5. Replace magic numbers with named constants.
6. Avoid encodings. Don't append prefixes or type information.

Functions rules

1. Small.
2. Do one thing.
3. Use descriptive names.
4. Prefer fewer arguments.
5. Have no side effects.

Bad	Good

total = 0
 	
def add_to_total(x):
  global total
  total += x # side effect
	def add(a, b):
	  return a + b # no side effect

6. Don't use flag arguments. Split method into several independent methods that can be called from the client without the flag.

Comments rules

1. Always try to explain yourself in code.
2. Don't be redundant.
3. Don't add obvious noise.
4. Don't use closing brace comments.
5. Don't comment out code. Just remove.
6. Use as explanation of intent.
7. Use as clarification of code.
8. Use as a warning of consequences.

Source code structure

1. Separate concepts vertically. Separate ***different*** ideas with blank lines.
2. Related code should appear vertically dense.
3. Declare variables close to their usage.
4. Dependent functions should be close.
5. Similar functions should be close.

6. Place functions in the downward direction. Functions should be ordered from high-level (more abstract) to low-level (more detailed). This reads like a top-down narrative.

```
def process_order(order):
    validate(order)
    charge(order)
    ship(order)

def validate(order): ...
def charge(order): ...
def ship(order): ...
```

7. Keep lines short.
8. Don't use horizontal alignment.
9. Use white space to associate related things and disassociate weakly related.
10. Don't break the indentation.

Tests

1. One assert per test.
2. Readable.
3. Fast.
4. Independent.
5. Repeatable.

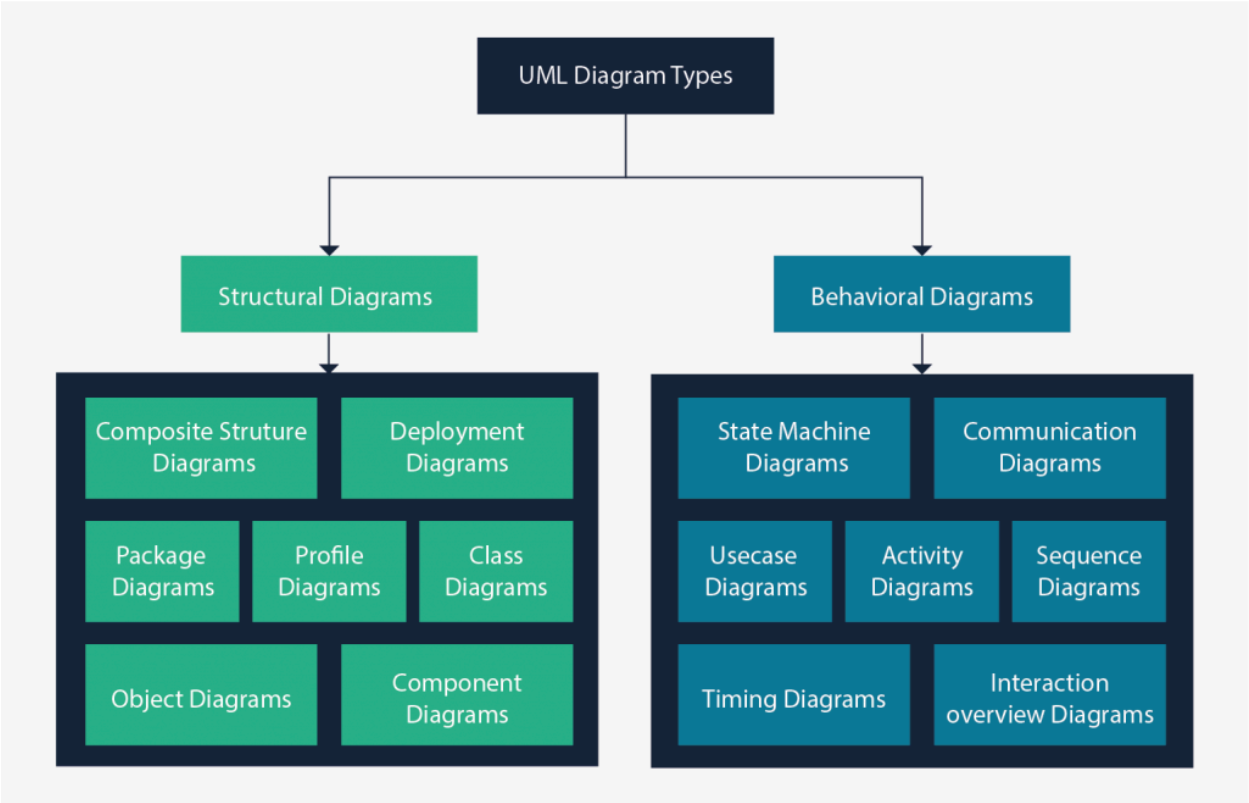
Code smells

1. Rigidity. The software is difficult to change. A small change causes a cascade of subsequent changes.
2. Fragility. The software breaks in many places due to a single change.
3. Immobility. You cannot reuse parts of the code in other projects because of involved risks and high effort.
4. Needless Complexity.
5. Needless Repetition.
6. Opacity. The code is hard to understand.

UML Diagram

https://creately.com/blog/diagrams/uml-diagram-types-examples/

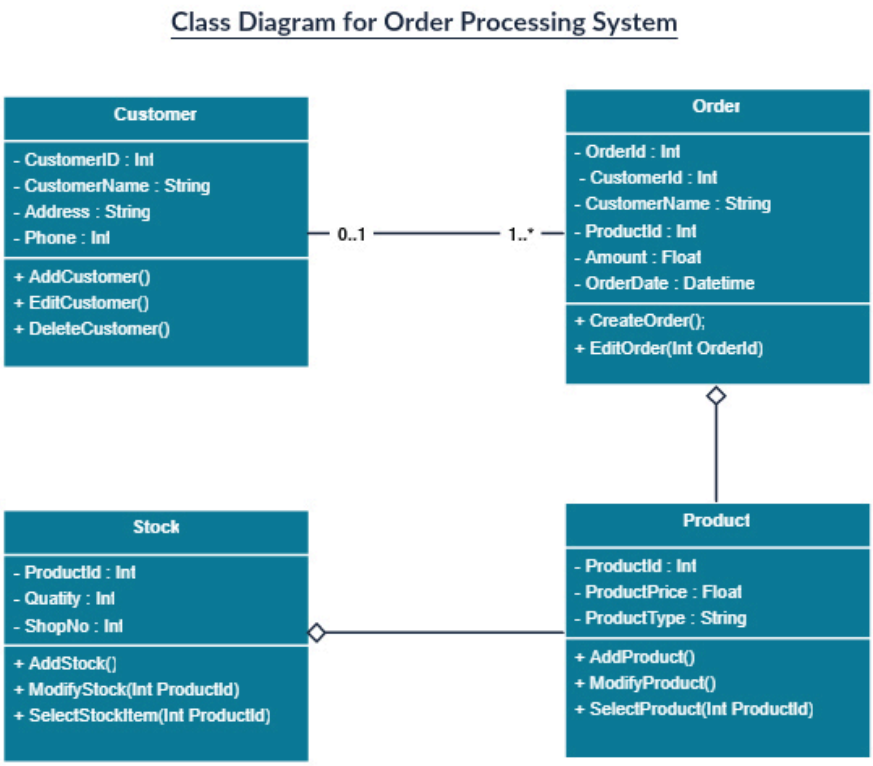
UML stands for Unified Modeling Language



Structure diagrams show the things in the modeled system. In a more technical term, they show different objects in a system. **Behavioral diagrams** show what should happen in a system.

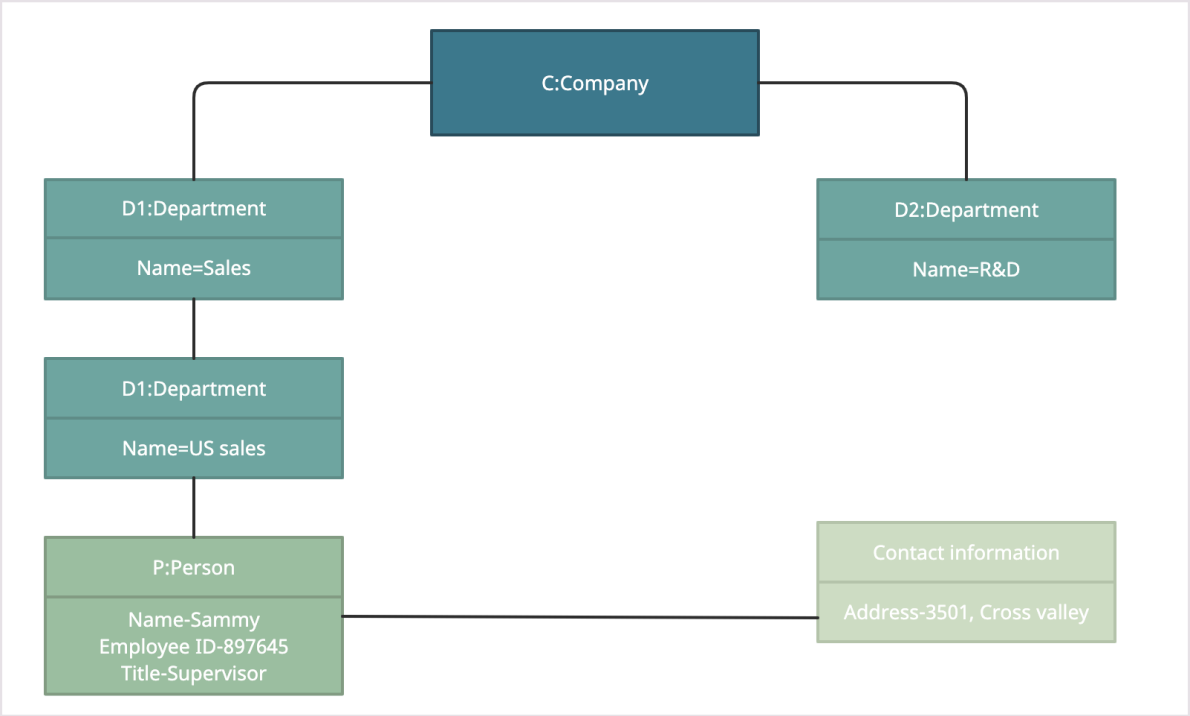
Class Diagram

Class diagrams are the main building block of any object-oriented solution. It shows the classes in a system, attributes, and operations of each class and the relationship between each class.



Object Diagram

Object Diagrams, sometimes referred to as Instance diagrams are very similar to class diagrams. Like class diagrams, they also show the relationship between objects but they use **real-world examples.** They show what a system will look like at a given time. Because there is data available in the objects, they are used to explain complex relationships between objects.



Component Diagram

A component diagram displays the structural relationship of components of a software system. These are mostly used when working with complex systems with many components. Components communicate with each other using **interfaces**. The interfaces are linked using connectors.
Parts:

1. **Component**

   Represent modular parts of the system that encapsulate functionalities.

2. **Interfaces**

   Define how components communicate with each other, ensuring that components can be developed and maintained independently.

3. **Relationships**

Lines and arrows.

- Dependency (dashed arrow): Indicates that one component relies on another.
- Association (solid line): Shows a more permanent relationship between components.

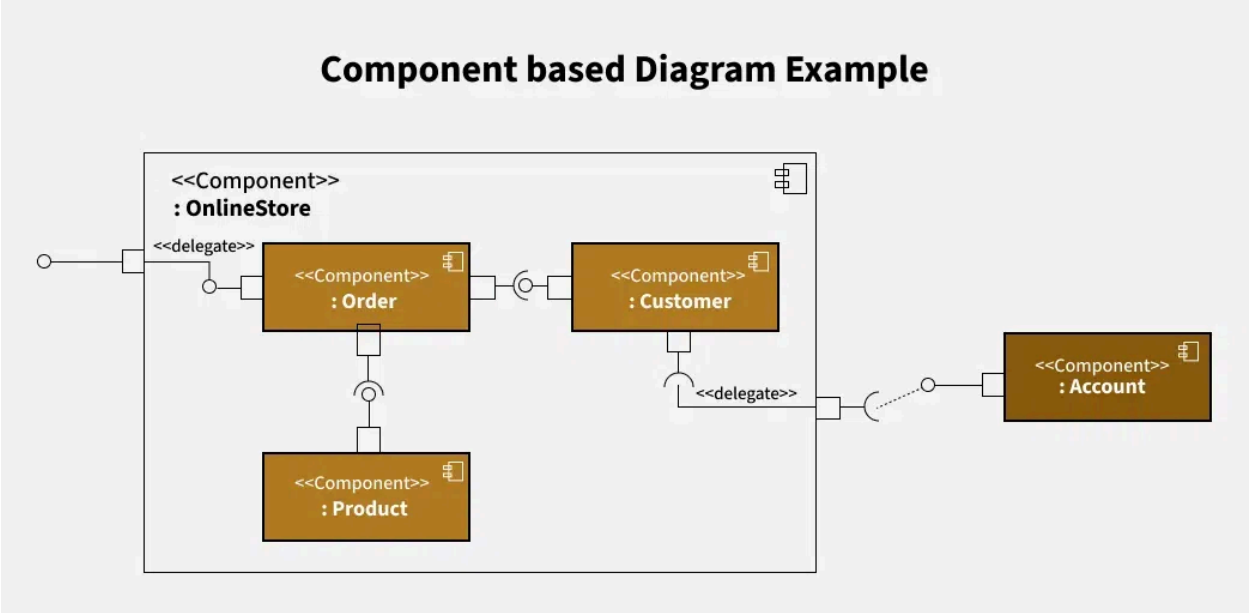
- Assembly connector: Connects a required interface of one component to a provided interface of another.

4. **Ports**

   Allow for more precise specification of interaction points, facilitating detailed design and implementation.

5. **Artifacts**

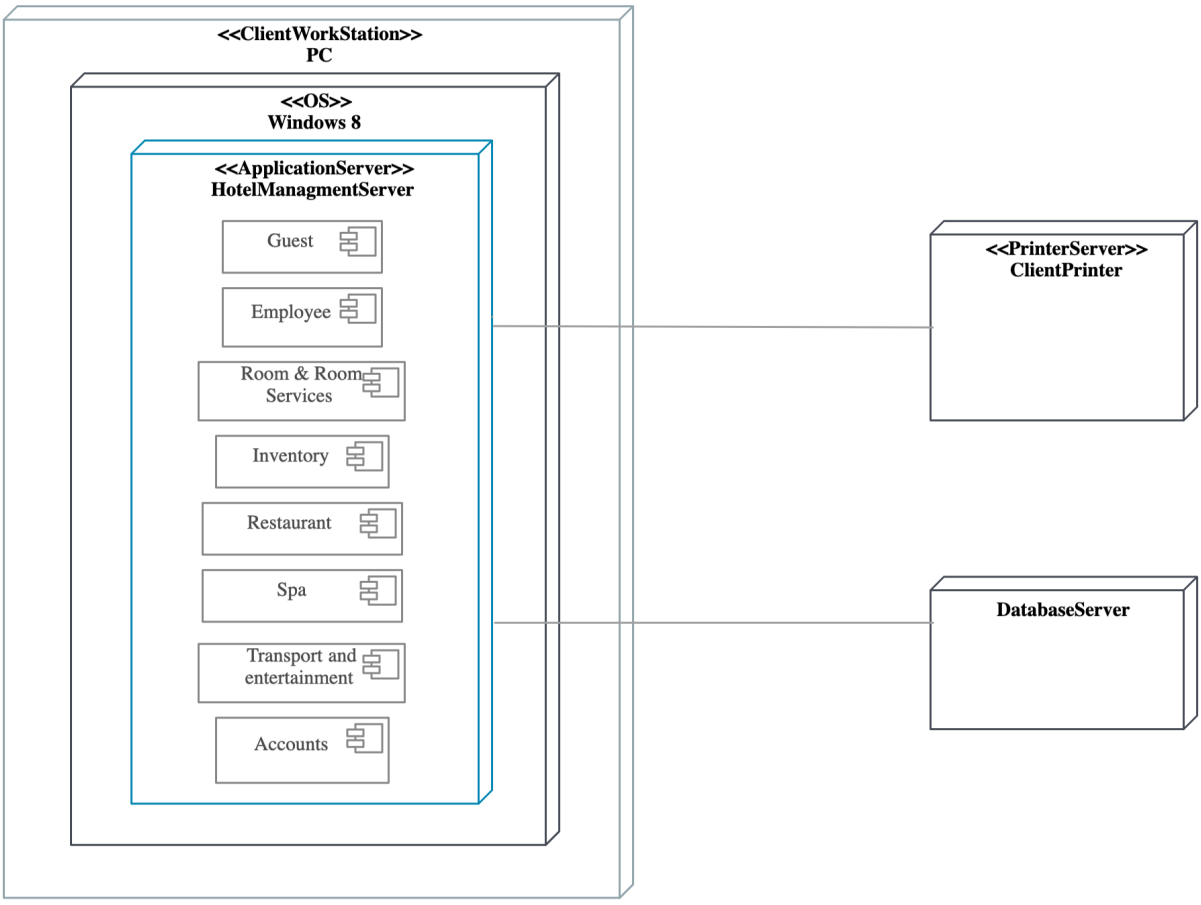
   Show how software artifacts, like executables or data files, relate to the components.



Deployment Diagram

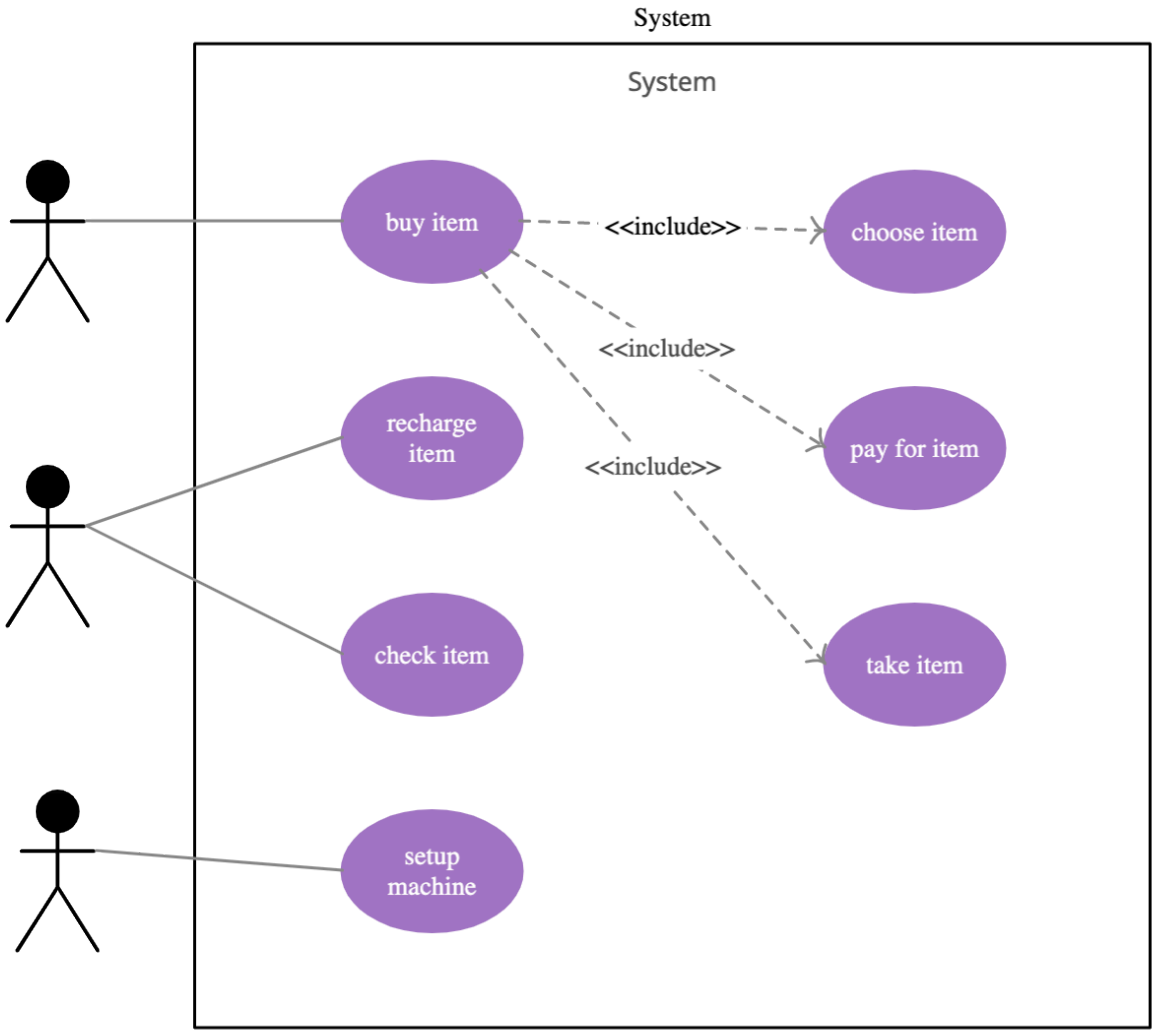
A deployment diagram shows the hardware of your system and the software in that hardware. Deployment diagrams are useful when your software solution is **deployed across multiple machines** with each having a unique configuration. Below is an example deployment diagram.

HOTEL MANAGEMENT SYSTEM



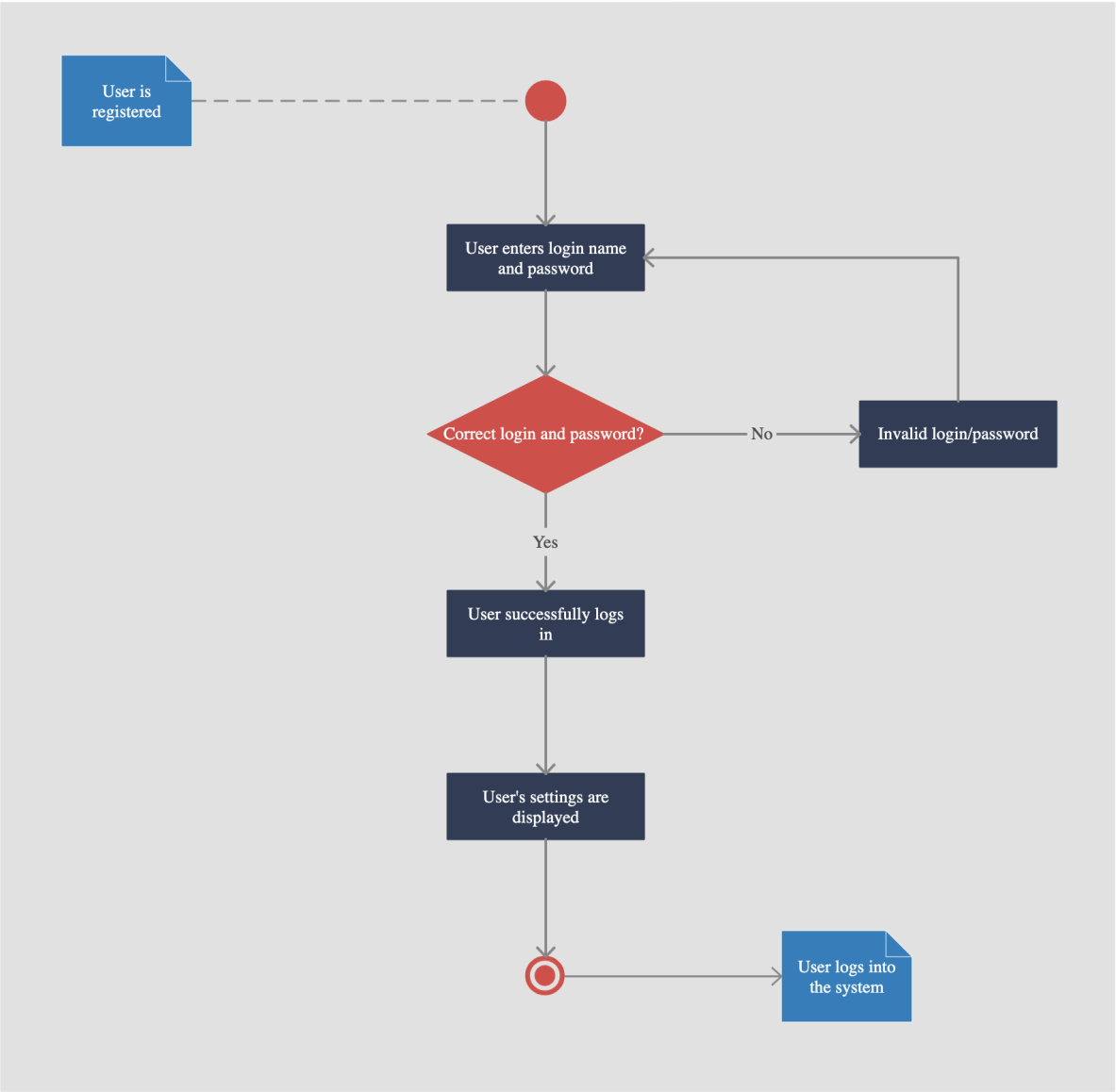
Use Case Diagram

As the most known diagram type of the behavioral UML types, Use case diagrams give a graphic overview of the actors involved in a system, different functions needed by those actors and how these different functions interact.



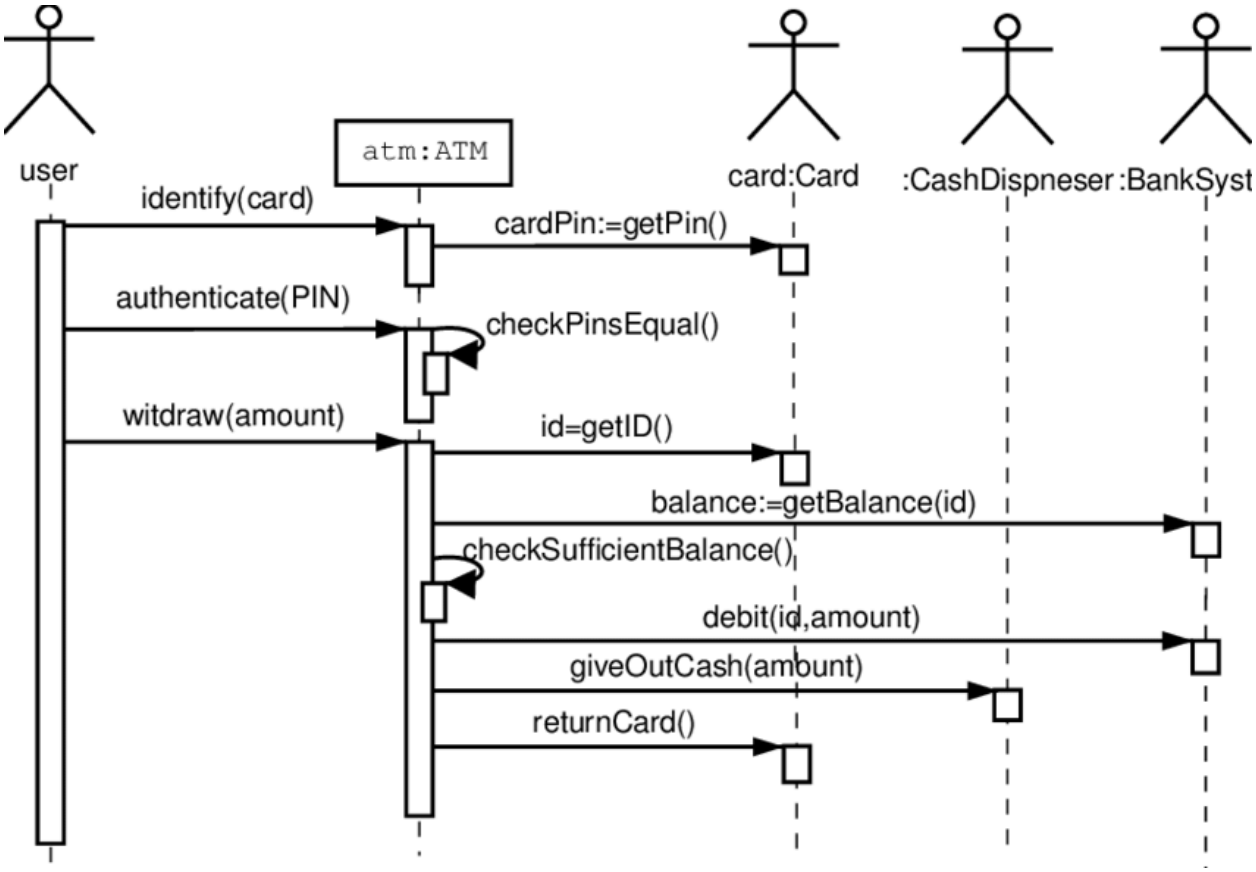
Activity Diagram

Activity diagrams represent workflows in a graphical way. They can be used to describe the business workflow or the operational workflow of any component in a system.



Sequence Diagram

Sequence diagrams in UML show how objects interact with each other and the order those interactions occur. **It's important to note that they show the interactions for a particular scenario.**



UML Diagram	Who Creates It	When It's Used	Why It's Useful (Purpose)
Class	Software architects, designers	Early design phase	To define the **structure** of classes, attributes, methods, and relationships
Object	Designers, developers	After class diagram; during runtime modeling	To show **instances** (objects) and how they relate at a specific moment
Component	Architects, dev leads	During high-level system design	To model **physical components** (e.g., modules, libraries, services) and dependencies
Deployment	DevOps, architects	Deployment planning	To show **hardware and software environment**, e.g., servers, containers, networks
Use Case	Business analysts, product owners	Early requirements phase	To capture **functional requirements** and interactions between users

			and the system
Activity	Analysts, developers	To model workflows or algorithmic logic	To show **process flows**, decisions, parallel actions (like a flowchart)
Sequence	Designers, developers	To specify behavior of use cases or methods	To model **object interactions over time** — who calls whom, in what order

Diagram	**Requirement**	**Design**	**Implementation/Deployment**
Class			
Object			
Component			
Deployment			
Use Case			
Activity			
Sequence			

Who Benefits?

- **Product Owners / Analysts** → Use case diagrams to define scope.
- **Architects** → Class, component, deployment diagrams for structure.
- **Developers** → Sequence, object, activity diagrams to guide implementation and logic.
- **DevOps** → Deployment diagrams to configure environments.